

ShapeLoader - Complete Documentation

Table of Contents

1. [Overview](#)
 2. [Architecture Decision: Service vs Static](#)
 3. [How ShapeLoader Works](#)
 4. [Configuration System](#)
 5. [Export Patterns](#)
 6. [Import vs Fetch](#)
 7. [Path Resolution](#)
 8. [Complete Flow Diagrams](#)
 9. [Best Practices](#)
 10. [Troubleshooting](#)
-

Overview

ShapeLoader is a dynamic shape loading system that:

- Loads shape class files (**.js**) and configuration files (**.json**)
- Supports both built-in and custom shapes
- Tracks loading state (loaded, failed, loading)
- Validates shapes before registration
- Handles errors gracefully

Key Features:

-  Dynamic ES6 module imports
 -  JSON configuration loading
 -  Parallel shape loading
 -  Comprehensive error tracking
 -  Debug logging support
-

Architecture Decision: Service vs Static

Question: Should ShapeLoader be registered in ServiceContainer?

Option 1: Register as Service  (Recommended)

```
// In ServiceProvider.js
container.register("shapeRegistry", () => new ShapeRegistry(), true);

// ShapeLoader is NOT registered - used directly
const loader = new ShapeLoader(shapeRegistry);
await loader.loadBuiltInShapes();
```

Why NOT register ShapeLoader:

- ❌ Only used during initialization
- ❌ No need for singleton pattern
- ❌ Not shared across components
- ❌ Creates unnecessary dependency

What to register:

- ✅ **ShapeRegistry** - Needs to be shared across app
- ❌ **ShapeLoader** - Use directly when needed
- ❌ **ShapeBuilder** - Factory pattern, not singleton
- ❌ **PresetManager** - Load presets once, pass as data
- ❌ **Shape Helpers** - Static utilities

Final ServiceContainer for Shapes

```
// Only ONE shape-related service
ServiceProvider.register(container) {
    container.register('shapeRegistry', () => new ShapeRegistry(), true);
    // That's it! Everything else is used directly.
}
```

Question: Should loadBuiltInShapes be static or instance method?

Answer: Keep as Instance Method ✅

```
// ✅ GOOD - Instance method (current approach)
const loader = new ShapeLoader(shapeRegistry);
await loader.loadBuiltInShapes(); // Built-in shapes
await loader.loadCustomShapes(); // Custom shapes
await loader.reloadShape(); // Reload during dev

// State tracking works across all methods
console.log(loader.getStats());
```

Why instance method is better:

- ✅ Consistent API for all loading operations
- ✅ State tracking across all loads
- ✅ Single configuration (validateOnLoad, throwOnError)
- ✅ Unified error handling
- ✅ Works for both built-in and custom shapes

How ShapeLoader Works

The Complete Loading Flow

COMPLETE SHAPELOADER FLOW

[1] main.js Initialization

```
└─> const loader = new ShapeLoader(shapeRegistry, options)
    └─> await loader.loadBuiltInShapes('/src/shapes/library')
```

[2] loadBuiltInShapes(basePath)

```
└─> Creates libraryConfig = {
    basic: [
      { type: "basic-circle", modulePath: "...", configPath: "..." },
      { type: "basic-rect", modulePath: "...", configPath: "..." }
    ],
    flowchart: [...],
    network: [...]
  }
└─> Calls loadLibrary(libraryConfig)
```

[3] loadLibrary(libraryConfig)

```
└─> For each category in libraryConfig:
    └─> loadCategory(category, shapes)
```

[4] loadCategory(category, shapes)

```
└─> Promise.all(shapes.map(shape =>
    └─> loadShape(type, modulePath, configPath)
  ))
```

[5] loadShape(type, modulePath, configPath)

```
└─> Check: Already loaded? → Return true
└─> Check: Currently loading? → Return false
└─> [A] Load Shape Class File
    └─> const module = await import(modulePath)
        └─> Get: CircleShape class constructor
└─> [B] Load Config JSON
    └─> const response = await fetch(configPath)
        └─> const config = await response.json()
            └─> Get: { id, name, defaultSize, defaultStyle, ... }
```

```

└─> [C] Validate (optional)
    └─> _validateShape(type, ShapeClass, config)
        └─> Check: class is function
        └─> Check: config has required fields
        └─> Check: defaultSize/defaultStyle valid
└─> [D] Register
    └─> registry.register(type, ShapeClass, config)
        └─> ShapeRegistry stores:
            {
              type: 'basic-circle',
              ShapeClass: CircleShape,
              config: {...},
              createInstance: () => new CircleShape(config)
            }

```

Function Call Hierarchy

```

loadBuiltInShapes()
└─> loadLibrary()
    └─> loadCategory('basic', shapes)
        └─> Promise.all([
            loadShape('basic-circle', '...', '...'),
            loadShape('basic-rect', '...', '...'),
            loadShape('basic-diamond', '...', '...')
          ])
    └─> loadCategory('flowchart', shapes)
        └─> Promise.all([
            loadShape('flowchart-process', '...', '...'),
            loadShape('flowchart-decision', '...', '...'),
            ...
          ])
    └─> loadCategory('network', shapes)
        └─> Promise.all([...])
└─> Returns: {
  totalCategories: 3,
  totalShapes: 12,
  loaded: 12,
  failed: 0,
  categories: {...}
}

```

Configuration System

Config JSON Structure

Location: `/src/shapes/library/basic/circle/config.json`

```
{
  "id": "basic-circle",
  "name": "Circle",
  "category": "basic",
  "description": "Basic circle shape",

  "defaultSize": {
    "width": 100,
    "height": 100
  },

  "defaultStyle": {
    "fill": "#FFFFFF",
    "stroke": "#000000",
    "strokeWidth": 2,
    "opacity": 1
  },

  "defaultPorts": [
    {
      "id": "top",
      "side": "top",
      "position": 0.5,
      "type": "both"
    },
    {
      "id": "right",
      "side": "right",
      "position": 0.5,
      "type": "both"
    },
    {
      "id": "bottom",
      "side": "bottom",
      "position": 0.5,
      "type": "both"
    },
    {
      "id": "left",
      "side": "left",
      "position": 0.5,
      "type": "both"
    }
  ]
},
```

```
"constraints": {  
  "minWidth": 20,  
  "minHeight": 20,  
  "maintainAspectRatio": true  
}  
}
```

How Config is Used in Shape Files

Shape Class: `/src/shapes/library/basic/circle/CircleShape.js`

```
import { BaseShape } from "../../../base/BaseShape.js";  
  
export class CircleShape extends BaseShape {  
  constructor(config) {  
    super(config); // Pass config to BaseShape  
  
    // Config is now available as instance properties:  
    // this.id = "basic-circle"  
    // this.name = "Circle"  
    // this.category = "basic"  
    // this.defaultSize = { width: 100, height: 100 }  
    // this.defaultStyle = { fill: "#FFF", stroke: "#000" }  
    // this.defaultPorts = [...]  
    // this.constraints = { minWidth: 20, ... }  
  }  
  
  /**  
   * Render the circle shape  
   */  
  render(x, y, width, height, styleOverride = {}) {  
    const g = document.createElementNS("http://www.w3.org/2000/svg", "g");  
  
    // Merge config's defaultStyle with runtime style  
    const finalStyle = { ...this.defaultStyle, ...styleOverride };  
  
    // Calculate circle parameters  
    const cx = x + width / 2;  
    const cy = y + height / 2;  
    const r = Math.min(width, height) / 2;  
  
    // Create circle element  
    const circle = document.createElementNS(  
      "http://www.w3.org/2000/svg",  
      "circle"  
    );  
    circle.setAttribute("cx", cx);  
    circle.setAttribute("cy", cy);  
    circle.setAttribute("r", r);
```

```
// Apply styles from config (merged with override)
circle.setAttribute("fill", finalStyle.fill);
circle.setAttribute("stroke", finalStyle.stroke);
circle.setAttribute("stroke-width", finalStyle.strokeWidth);
circle.setAttribute("opacity", finalStyle.opacity);

g.appendChild(circle);
return g;
}

/**
 * Calculate port positions based on config
 */
getPortPositions(x, y, width, height) {
  const cx = x + width / 2;
  const cy = y + height / 2;
  const r = Math.min(width, height) / 2;

  // Use defaultPorts from config
  return this.defaultPorts.map((port) => {
    let px, py;

    switch (port.side) {
      case "top":
        px = cx;
        py = cy - r;
        break;
      case "right":
        px = cx + r;
        py = cy;
        break;
      case "bottom":
        px = cx;
        py = cy + r;
        break;
      case "left":
        px = cx - r;
        py = cy;
        break;
    }

    return {
      ...port, // Include all config from defaultPorts
      x: px,
      y: py,
    };
  });
}

export default CircleShape;
```

BaseShape Class

```
// src/shapes/base/BaseShape.js

export class BaseShape {
  constructor(config) {
    // Store all config properties as instance variables
    this.id = config.id;
    this.name = config.name;
    this.category = config.category;
    this.description = config.description || "";

    // Size configuration
    this.defaultSize = config.defaultSize || { width: 100, height: 100 };

    // Style configuration
    this.defaultStyle = config.defaultStyle || {
      fill: "#FFFFFF",
      stroke: "#000000",
      strokeWidth: 2,
      opacity: 1,
    };

    // Ports configuration
    this.defaultPorts = config.defaultPorts || [];

    // Constraints
    this.constraints = config.constraints || {
      minWidth: 10,
      minHeight: 10,
      maintainAspectRatio: false,
    };
  }

  /**
   * Must be overridden by subclass
   */
  render(x, y, width, height, style) {
    throw new Error("render() must be implemented by subclass");
  }

  /**
   * Get default width from config
   */
  getDefaultWidth() {
    return this.defaultSize.width;
  }

  /**
   * Get default height from config
   */
}
```



```

getDefaultHeight() {
    return this.defaultSize.height;
}

/**
 * Get default style from config
 */
getDefaultStyle() {
    return { ...this.defaultStyle };
}

/**
 * Validate dimensions against constraints
 */
validateSize(width, height) {
    if (width < this.constraints.minWidth) {
        return { valid: false, error: "Width too small" };
    }
    if (height < this.constraints.minHeight) {
        return { valid: false, error: "Height too small" };
    }
    return { valid: true };
}
}

```

Config Data Flow

CONFIG DATA FLOW

[1] JSON Config File

```

↓
{
  "id": "basic-circle",
  "name": "Circle",
  "defaultSize": { "width": 100, "height": 100 },
  "defaultStyle": { "fill": "#FFF", "stroke": "#000" },
  "defaultPorts": [...]
}

```

[2] ShapeLoader.loadShape()

```

↓
fetch('/path/to/config.json')
↓
config = { id: "basic-circle", name: "Circle", ... }

```

[3] ShapeRegistry.register(type, ShapeClass, config)

```

↓
Stores:

```

```
{
  type: 'basic-circle',
  ShapeClass: CircleShape,
  config: { ... }, ← JSON config stored here
  createInstance: () => new CircleShape(config)
}
```

[4] User creates a node

```
↓
const shapeDef = registry.getShape('basic-circle')
const shape = shapeDef.createInstance()
↓
Calls: new CircleShape(config)
```

[5] CircleShape constructor

```
↓
constructor(config) {
  super(config); // Pass to BaseShape
  // Now this.defaultSize = config.defaultSize
  // Now this.defaultStyle = config.defaultStyle
  // Now this.defaultPorts = config.defaultPorts
}
```

[6] Rendering with config + overrides

```
↓
shape.render(200, 150, 120, 120, { fill: '#FF0000' })
↓
Inside render():
  const finalStyle = {
    ...this.defaultStyle, // From config: { fill: "#FFF", stroke:
"#000" }
    ...styleOverride      // Override: { fill: '#FF0000' }
  }
  // Result: { fill: '#FF0000', stroke: "#000" }
↓
Creates SVG with merged style
```

Key Points:

1. **Config loaded once** during initialization
2. **Config stored in registry** alongside class
3. **Config passed to constructor** when creating instance
4. **Config provides defaults** that can be overridden at runtime

Export Patterns

Question: What if JS file has multiple classes? Which is default?

Pattern 1: Default Export (Recommended)

```
// CircleShape.js

class CircleShape {
  constructor() {}
  render() {}
}

// Only ONE default export per file
export default CircleShape;
```

Importing:

```
// Static import
import CircleShape from "./CircleShape.js";

// Dynamic import
const module = await import("./CircleShape.js");
const CircleShape = module.default; // ✅ Access via .default
```

Pattern 2: Named Export

```
// CircleShape.js

export class CircleShape {
  // ← Named export
  constructor() {}
  render() {}
}
```

Importing:

```
// Static import
import { CircleShape } from "./CircleShape.js";

// Dynamic import
const module = await import("./CircleShape.js");
const CircleShape = module.CircleShape; // ✅ Access by name
```

Pattern 3: Multiple Classes in One File

```
// BasicShapes.js
```

```
export class CircleShape {
  constructor() {}
}

export class RectShape {
  constructor() {}
}

export class TriangleShape {
  constructor() {}
}
```

Importing:

```
// Static import
import { CircleShape, RectShape } from "./BasicShapes.js";

// Dynamic import
const module = await import("./BasicShapes.js");
const CircleShape = module.CircleShape;
const RectShape = module.RectShape;
```

Pattern 4: Mix of Default + Named

```
// CircleShape.js

class CircleShape {
  constructor() {}
}

// Helper functions as named exports
export function calculateRadius(width, height) {
  return Math.min(width, height) / 2;
}

export function validateCircle(width, height) {
  return width === height;
}

// Main class as default export
export default CircleShape;
```

Importing:

```
// Static import
import CircleShape, { calculateRadius, validateCircle } from
```

```
"/CircleShape.js";

// Dynamic import
const module = await import("/CircleShape.js");
const CircleShape = module.default; // Main class
const calculateRadius = module.calculateRadius; // Helper
```

How ShapeLoader Handles Both

```
async loadShape(type, modulePath, configPath) {
  const shapeModule = await import(modulePath);

  // Try default export first, then named export
  const ShapeClass =
    shapeModule.default || // export default
    shapeModule[this._getShapeClassName(type)]; // export class
  CircleShape

  if (!ShapeClass) {
    throw new Error(`Shape class not found in module: ${modulePath}`);
  }
}
```

This handles:

1. ☒ export default CircleShape → shapeModule.default
2. ☒ export class CircleShape → shapeModule.CircleShape
3. ☒ Both patterns work seamlessly

The _getShapeClassName Helper

```
_getShapeClassName(type) {
  // Convert 'flowchart-process' to 'ProcessShape'
  const parts = type.split("-"); // ['flowchart', 'process']
  const className = parts[parts.length - 1] // 'process'
    .split("_")
    .map((part) => part.charAt(0).toUpperCase() + part.slice(1)) //
    'Process'
    .join("");

  return `${className}Shape`; // 'ProcessShape'
}
```

Examples:

```
_getShapeClassName("basic-circle"); // → 'CircleShape'
_getShapeClassName("flowchart-process"); // → 'ProcessShape'
_getShapeClassName("network-router"); // → 'RouterShape'
_getShapeClassName("uml-class_diagram"); // → 'ClassDiagramShape'
```

Export Pattern Comparison

Export Style	Syntax	Access After Import	Use Case
Default	<code>export default CircleShape</code>	<code>module.default</code>	✔ One class per file (recommended)
Named	<code>export class CircleShape</code>	<code>module.CircleShape</code>	Multiple classes per file
Mixed	Both default + named	Both available	Main class + helpers

Recommendation

Use default export for shape classes:

```
// CircleShape.js
class CircleShape {
  constructor(config) {
    this.config = config;
  }
  render() {}
}

export default CircleShape; // ✔ Recommended
```

Why?

- ✔ ShapeLoader tries `module.default` first (faster)
- ✔ Clear one-to-one mapping: file → shape
- ✔ Standard ES6 module pattern
- ✔ Easy to maintain

Import vs Fetch

Question: Why use `import()` for JS and `fetch()` for JSON?

Comparison: Static Import vs Dynamic Import vs Fetch

Feature	Static <code>import</code>	Dynamic <code>import()</code>	<code>fetch()</code>
---------	----------------------------	-------------------------------	----------------------

Feature	Static <code>import</code>	Dynamic <code>import()</code>	<code>fetch()</code>
When runs	File load time	Runtime (when called)	Runtime (when called)
Path	Must be literal	Can be variable	Can be variable
Returns	Module exports	Promise<Module>	Promise<Response>
For JS files	✅ Yes	✅ Yes	❌ No (text only)
For JSON	⚠ Limited	⚠ Experimental	✅ Yes (best)
Conditional	❌ No	✅ Yes	✅ Yes
Async	No (blocks)	Yes	Yes

Static Import (Top of File)

```
// At the top of the file
import { CircleShape } from './CircleShape.js';

// ✅ Runs immediately when file loads
// ✅ Module available synchronously
// ❌ Cannot use variables in path
// ❌ Cannot conditionally import
// ❌ Path must be string literal

const shape = new CircleShape(); // Available immediately
```

Dynamic Import (Runtime)

```
// Inside a function
async loadShape() {
  const modulePath = './CircleShape.js'; // ✅ Can be variable!

  const module = await import(modulePath); // ✅ Loads at runtime
  const CircleShape = module.default;

  const shape = new CircleShape();
}

// ✅ Loads at runtime (when function called)
// ✅ Can use variables in path
// ✅ Can conditionally import
// ✅ Returns a Promise (async)
```

Why Not Both `import()`?

You CAN import JSON, but it has issues:

```
// ⚠ This works but is experimental
const configModule = await import("./config.json", {
  assert: { type: "json" },
});
const config = configModule.default;
```

Problems:

- ❌ Not supported in all browsers yet
 - ❌ Requires JSON import assertions (experimental)
 - ❌ More complex syntax
 - ✅ `fetch()` is universally supported
-

Why Not Both `fetch()`?

You CANNOT use `fetch()` for JavaScript:

```
// ❌ This gets JS as TEXT, not executable code
const response = await fetch("./CircleShape.js");
const jsCode = await response.text();

console.log(jsCode); // "export class CircleShape { ... }"

// ❌ Cannot execute this as a class
const shape = new jsCode.CircleShape(); // ERROR!
```

To make it work requires `eval()`:

- ❌ Dangerous (security risk)
 - ❌ Doesn't support ES6 modules
 - ❌ Very bad practice
-

What `import()` and `fetch()` Return

`import()` Returns `Promise<Module>`

```
const modulePromise = import("./CircleShape.js");
// Type: Promise<Module>

// When resolved:
const module = await modulePromise;
console.log(module);
// {
```



```
// default: CircleShape,      // export default CircleShape
// someFunction: function,    // export function someFunction()
// someVariable: 42           // export const someVariable = 42
// }
```

fetch() Returns Promise<Response>

```
const responsePromise = fetch("./config.json");
// Type: Promise<Response>

// When resolved:
const response = await responsePromise;
console.log(response);
// Response {
//   ok: true,
//   status: 200,
//   statusText: "OK",
//   headers: Headers { ... },
//   body: ReadableStream,
//   json: function,      ← Need to call this
//   text: function,
//   blob: function
// }

// Extract the data (also returns Promise!)
const config = await response.json();
console.log(config);
// { id: "circle", name: "Circle", ... }
```

ShapeLoader Uses Both

```
async loadShape(type, modulePath, configPath) {
  // =====
  // IMPORT – For JavaScript modules
  // =====
  const shapeModule = await import(modulePath);
  // Returns: { default: CircleShape, ... }
  const ShapeClass = shapeModule.default;
  // ShapeClass is executable constructor function

  // =====
  // FETCH – For JSON data
  // =====
  const response = await fetch(configPath);
  // Returns: Response { ok: true, status: 200, ... }
  const config = await response.json();
  // Returns: { id: "circle", name: "Circle", ... }
```

```
// config is plain JavaScript object

// =====
// USE TOGETHER
// =====
const instance = new ShapeClass(config);
}
```

Are They Async?

Yes! Both return Promises:

```
// import() returns Promise
const promise1 = import("./CircleShape.js");
console.log(promise1); // Promise { <pending> }

// fetch() returns Promise
const promise2 = fetch("./config.json");
console.log(promise2); // Promise { <pending> }

// Both need await to resolve
const module = await promise1; // { default: CircleShape }
const response = await promise2; // Response { ok: true, ... }
const config = await response.json(); // { id: "circle", ... }
```

Promise Resolution Methods

Method 1: **async/await** (Recommended)

```
async loadShape() {
  try {
    // import() with await
    const module = await import('./CircleShape.js');
    const CircleShape = module.default;

    // fetch() with await
    const response = await fetch('./config.json');
    const config = await response.json();

    return new CircleShape(config);
  } catch (error) {
    console.error('Failed:', error);
  }
}
```

Method 2: `.then()` chains

```
loadShape() {
  return import('./CircleShape.js')
    .then(module => {
      const CircleShape = module.default;
      return fetch('./config.json')
        .then(response => response.json())
        .then(config => new CircleShape(config));
    })
    .catch(error => {
      console.error('Failed:', error);
    });
}
```

Method 3: `Promise.all()` (Parallel - Faster!)

```
async loadShape() {
  // Load both simultaneously
  const [module, response] = await Promise.all([
    import('./CircleShape.js'),
    fetch('./config.json')
  ]);

  const CircleShape = module.default;
  const config = await response.json();

  return new CircleShape(config);
}
```

Summary: Why This Combination?

Operation	Use	Why
Load <code>.js</code> file	<code>import()</code>	Gets executable code (classes/functions)
Load <code>.json</code> file	<code>fetch()</code>	Gets data (configuration objects)

Best practice:

- ✓ `await import()` for JavaScript modules
- ✓ `await fetch()` for JSON/data files
- ✓ Both are async and return Promises
- ✓ Use `await` to unwrap the Promise

Path Resolution

Question: Why are paths wrong?

The Problem

```
Error: Failed to fetch
http://localhost:8000/src/shapes/loader/library/flowchart/process/ProcessShape.js
```

^^^^^^

WRONG!

Actual location:

```
http://localhost:8000/src/shapes/library/flowchart/process/ProcessShape.js
```

^^^^^^

CORRECT

Root Cause

Relative paths like `./library` resolve **relative to the importing file's directory**, not the project root.

```
// ShapeLoader.js is at: /src/shapes/loader/ShapeLoader.js

async loadBuiltInShapes(basePath = "./library") {
  // "./library" resolves relative to ShapeLoader.js location
  // Result: /src/shapes/loader/library ❌ WRONG!
}
```

Solution: Use Absolute Paths

```
// ✅ Option 1: Pass absolute path when calling
const loader = new ShapeLoader(shapeRegistry);
await loader.loadBuiltInShapes('/src/shapes/library'); // Leading slash!

// ✅ Option 2: Set absolute default in method
async loadBuiltInShapes(basePath = "/src/shapes/library") {
  // Absolute path from project root
}
```

Path Resolution Rules

Path Type	Example	Resolves To
-----------	---------	-------------

Path Type	Example	Resolves To
Absolute	/src/shapes/library	http://localhost:8000/src/shapes/library
Relative	./library	Relative to current file's directory
Relative	../library	One level up from current file

In ShapeLoader.js:

```
// File location: /src/shapes/loader/ShapeLoader.js

"./library"      → /src/shapes/loader/library  ❌
"../library"     → /src/shapes/library         ✅
"/src/shapes/library" → /src/shapes/library   ✅ (Best)
```

Complete Path Examples

```
// In main.js
const loader = new ShapeLoader(shapeRegistry);

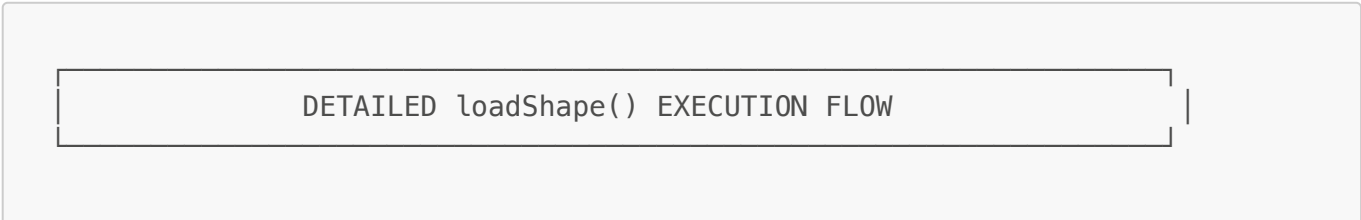
// ✅ GOOD – Absolute path
await loader.loadBuiltInShapes("/src/shapes/library");

// ❌ BAD – Relative path (depends on where ShapeLoader.js is)
await loader.loadBuiltInShapes("./library");

// After loading, the actual paths used:
const libraryConfig = {
  basic: [
    {
      type: "basic-circle",
      modulePath: "/src/shapes/library/basic/circle/CircleShape.js", // ✅
      configPath: "/src/shapes/library/basic/circle/config.json", // ✅
    },
  ],
};
```

Complete Flow Diagrams

Detailed loadShape() Flow



```
loadShape(type, modulePath, configPath)

  ↳ Log: "Enter loadShape()"

  ↳ [CHECK 1] this.loaded.has(type)?
    ↳ YES → Log: "Already loaded"
    ↳ Return true

  ↳ [CHECK 2] this.loading.has(type)?
    ↳ YES → Log: "Currently loading"
    ↳ Return false

  ↳ this.loading.add(type) ← Mark as loading

  ↳ TRY {
    ↳ [STEP 1] Load Shape Class
      ↳ Log: "Importing module..."
      ↳ const module = await import(modulePath)
        ↳ Promise { <pending> }
        ↳ Resolves to: { default: CircleShape, ... }

      ↳ Extract class:
        const ShapeClass = module.default ||
                           module[_getShapeClassName(type)]

      ↳ if (!ShapeClass) throw Error

    ↳ [STEP 2] Load Config
      ↳ Log: "Loading config..."
      ↳ const response = await fetch(configPath)
        ↳ Promise { <pending> }
        ↳ Resolves to: Response { ok: true, status: 200 }

      ↳ if (!response.ok) throw Error

      ↳ const config = await response.json()
        ↳ Promise { <pending> }
        ↳ Resolves to: { id: "circle", name: "Circle", ... }

      ↳ Config loaded

    ↳ [STEP 3] Validate (if enabled)
      ↳ if (this.validateOnLoad)
        ↳ Log: "Validating..."
        ↳ _validateShape(type, ShapeClass, config)
          ↳ Check: typeof ShapeClass === 'function'
          ↳ Check: config.id exists
          ↳ Check: config.name exists
          ↳ Check: config.category exists
```

[illegible]

Parallel Loading with Promise.all

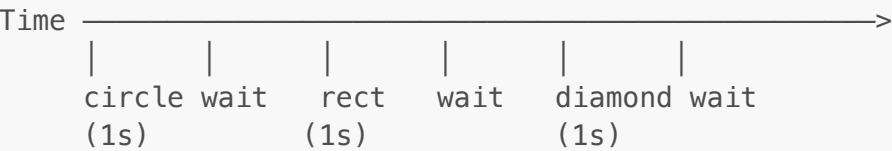
```
loadCategory('basic', [circle, rect, diamond])
  |
  └> Promise.all([
```

```

    |
    |-> loadShape('basic-circle', ...)
    |   |
    |   |-> await
import('/src/shapes/library/basic/circle/CircleShape.js')
    |   |-> await
fetch('/src/shapes/library/basic/circle/config.json')
    |
    |-> loadShape('basic-rect', ...)
    |   |
    |   |-> await
import('/src/shapes/library/basic/rect/RectShape.js')
    |   |-> await fetch('/src/shapes/library/basic/rect/config.json')
    |
    |-> loadShape('basic-diamond', ...)
    |   |
    |   |-> await
import('/src/shapes/library/basic/diamond/DiamondShape.js')
    |   |-> await
fetch('/src/shapes/library/basic/diamond/config.json')
  ])
  |
  |-> All three load simultaneously!
      Time = Max(circle, rect, diamond) ← Faster than sequential

```

SEQUENTIAL (Slow):



Total: 3 seconds

PARALLEL (Fast):



Total: 1 second (all at once!)

Complete Initialization Flow



Parallel
Parallel
Parallel

```

|> await loadCategory('basic', shapes)
|   ↳ Promise.all([
|       loadShape('basic-circle', ...), ←
|
|       loadShape('basic-rect', ...), ←
|
|       loadShape('basic-diamond', ...) ←
|
|   ])
|> await loadCategory('flowchart', shapes)
|   ↳ Promise.all([...])
|> await loadCategory('network', shapes)
|   ↳ Promise.all([...])

```

[2] Each loadShape() does:

```
|
|→ await import('/src/shapes/library/basic/circle/CircleShape.js')
|   ↳ Get: CircleShape class
|
|→ await fetch('/src/shapes/library/basic/circle/config.json')
|   ↳ Get: { id, name, defaultSize, defaultStyle, ... }
|
|↳ shapeRegistry.register('basic-circle', CircleShape, config)
|   ↳ ShapeRegistry stores:
|       {
|         type: 'basic-circle',
|         ShapeClass: CircleShape,
|         config: { ... },
|         createInstance: () => new CircleShape(config)
|       }
```

[3] Result:

```
|
|↳ ShapeRegistry now contains all shapes:
|   ↳ basic-circle
|   ↳ basic-rect
|   ↳ basic-diamond
|   ↳ flowchart-process
|   ↳ flowchart-decision
|   ↳ network-server
|   ↳ ... (all shapes loaded)
```

[4] App ready to use shapes:

```
|
|↳ When user clicks "Circle" in palette:
|   ↳ const shapeDef = shapeRegistry.getShape('basic-circle')
|   ↳ const shape = shapeDef.createInstance()
|       ↳ Calls: new CircleShape(config)
|
|↳ const svg = shape.render(x, y, width, height, style)
|   ↳ Creates SVG using config defaults + style overrides
```

Best Practices

1. Always Use Absolute Paths

```
// ✅ GOOD
await loader.loadBuiltInShapes("/src/shapes/library");
```

```
// ❌ BAD
await loader.loadBuiltInShapes("./library");
```

2. Use Default Exports for Shape Classes

```
// ✅ GOOD
class CircleShape {}
export default CircleShape;

// ⚠️ OK but not recommended
export class CircleShape {}
```

3. Always Validate Configs

```
// ✅ GOOD
const loader = new ShapeLoader(shapeRegistry, {
  validateOnLoad: true, // Catch errors early
  throwOnError: false, // Continue loading other shapes
});
```

4. Use Parallel Loading

```
// ✅ GOOD - Shapes in same category load in parallel
await loadCategory("basic", shapes); // All shapes load together

// Already implemented in ShapeLoader via Promise.all()
```

5. Handle Errors Gracefully

```
// ✅ GOOD - Check results after loading
const results = await loader.loadBuiltInShapes("/src/shapes/library");

if (results.failed > 0) {
  console.error(`Failed to load ${results.failed} shapes:`,
    results.categories);
  // App can still work with shapes that loaded successfully
}
```

6. Use Debug Logging

```
// Enable debug logging
localStorage.setItem("debugMode", "true");

// See detailed logs:
// [ShapeLoader] Enter loadShape()
// [ShapeLoader] Importing module...
// [ShapeLoader] Loading config...
// [ShapeLoader] Successfully loaded: basic-circle
```

7. One Config Per Shape

```
// ✅ GOOD - Each shape has its own config.json
/shapes/library/basic/circle/config.json
/shapes/library/basic/rect/config.json

// ❌ BAD - Shared config makes customization hard
/shapes/library/basic/config.json (for all basic shapes)
```

8. Config Provides Defaults, Runtime Provides Overrides

```
// Config defines defaults
{
  "defaultStyle": {
    "fill": "#FFFFFF",
    "stroke": "#000000"
  }
}

// Runtime overrides specific values
shape.render(x, y, width, height, {
  fill: "#FF0000" // Override fill, keep other defaults
});

// Result: { fill: "#FF0000", stroke: "#000000" }
```

Troubleshooting

Problem 1: "Failed to fetch" 404 Errors

```
GET http://localhost:8000/src/shapes/loader/library/... 404
```

Cause: Relative path resolving incorrectly

Solution:

```
// Use absolute path
await loader.loadBuiltInShapes("/src/shapes/library"); // Leading slash!
```

Problem 2: "Shape class not found"

```
Error: Shape class not found in module:
/src/shapes/library/basic/circle/CircleShape.js
```

Causes:

1. No `export default` and no matching named export
2. Class name doesn't match `_getShapeClassName()` output

Solutions:

```
// ✅ Option 1: Use default export (recommended)
export default CircleShape;

// ✅ Option 2: Use named export matching the pattern
export class CircleShape {} // Must match _getShapeClassName('basic-circle')
```

Problem 3: Config Validation Fails

```
Error: Config missing required field: id
```

Cause: Config JSON missing required fields

Solution: Ensure config has all required fields:

```
{
  "id": "basic-circle", // Required
  "name": "Circle",     // Required
  "category": "basic",  // Required
  "defaultSize": { ... },
```

```
"defaultStyle": { ... }  
}
```

Problem 4: Shapes Load But Don't Render

Cause: Shape class doesn't implement `render()` method

Solution:

```
class CircleShape extends BaseShape {  
  // Must implement render()  
  render(x, y, width, height, style) {  
    // Create and return SVG element  
    return svgElement;  
  }  
}
```

Problem 5: Config Not Applied to Shape

Cause: Not passing config to parent constructor

Solution:

```
class CircleShape extends BaseShape {  
  constructor(config) {  
    super(config); // ← Don't forget this!  
  }  
}
```

Problem 6: Multiple Shapes Load Slowly

Cause: Sequential loading instead of parallel

Already Solved: ShapeLoader uses `Promise.all()` for parallel loading within categories

To verify:

```
// Check debug logs – all shapes in category should start loading together  
// [ShapeLoader] Importing module for 'basic-circle'  
// [ShapeLoader] Importing module for 'basic-rect'      ← Simultaneous  
// [ShapeLoader] Importing module for 'basic-diamond'   ← Simultaneous
```

Summary

Key Takeaways

1. **ShapeLoader is instance-based** - Not registered in ServiceContainer
 2. **Only ShapeRegistry is registered** - It's the only service needed
 3. **Config provides defaults** - Runtime provides overrides
 4. **Use absolute paths** - Avoid relative path issues
 5. **Both import() and fetch() are async** - Both return Promises
 6. **Use default exports** - Simplest and most reliable
 7. **Parallel loading is automatic** - Via `Promise.all()` in categories
 8. **Config flows through constructor** - BaseShape stores it as instance properties
-

Quick Reference

```
// 1. Register only ShapeRegistry
container.register("shapeRegistry", () => new ShapeRegistry(), true);

// 2. Use ShapeLoader directly
const loader = new ShapeLoader(shapeRegistry, { validateOnLoad: true });

// 3. Load with absolute path
const results = await loader.loadBuiltInShapes("/src/shapes/library");

// 4. Check results
console.log(`Loaded: ${results.loaded}, Failed: ${results.failed}`);

// 5. Use shapes
const shapeDef = shapeRegistry.getShape("basic-circle");
const shape = shapeDef.createInstance();
const svg = shape.render(x, y, width, height, { fill: "#FF0000" });
```

Architecture Overview

COMPLETE ARCHITECTURE

ServiceContainer

└> ShapeRegistry (singleton service)

ShapeLoader (not registered)

└> new ShapeLoader(shapeRegistry)

└> loadBuiltInShapes('/src/shapes/library')

└┬> For each shape:

└┴> import() → Get ShapeClass

```

└─> fetch() → Get config
└─> registry.register(type, ShapeClass, config)

```

ShapeRegistry

```
└─> Stores: { type, ShapeClass, config, createInstance() }
```

Shape Classes

```

└─> Extend BaseShape
└─> Receive config in constructor
└─> Store config as instance properties
└─> Use config in render() method

```

Runtime

```

└─> Get shape: registry.getShape(type)
└─> Create instance: shapeDef.createInstance()
└─> Render: shape.render(x, y, w, h, styleOverride)

```

Appendix: Complete Code Examples

Complete ShapeLoader.loadShape()

```

async loadShape(type, modulePath, configPath) {
  this.log.enter("loadShape", { type, modulePath, configPath });

  // Check if already loaded
  if (this.loaded.has(type)) {
    this.log.warn(`Shape '${type}' is already loaded`);
    this.log.exit("loadShape", true);
    return true;
  }

  // Check if currently loading
  if (this.loading.has(type)) {
    this.log.warn(`Shape '${type}' is currently being loaded`);
    this.log.exit("loadShape", false);
    return false;
  }

  this.loading.add(type);

  try {
    // Load shape class
    this.log.debug(`Importing module for '${type}' from '${modulePath}'`);
    const shapeModule = await import(modulePath);
    const ShapeClass =
      shapeModule.default || shapeModule[this._getShapeClassName(type)];

    if (!ShapeClass) {
      throw new Error(`Shape class not found in module: ${modulePath}`);
    }
  }

```



```

// Load config
this.log.debug(`Loading config for '${type}' from '${configPath}'`);
const config = await this._loadConfig(configPath);

// Validate if enabled
if (this.validateOnLoad) {
  this.log.debug(`Validating shape '${type}'...`);
  this._validateShape(type, ShapeClass, config);
}

// Register with registry
const success = this.registry.register(type, ShapeClass, config);
if (!success) {
  throw new Error(`ShapeRegistry failed to register shape '${type}'`);
}

// Mark as loaded
this.loaded.add(type);
this.loading.delete(type);
this.failed.delete(type);

this.log.info(`Successfully loaded and registered shape '${type}'`);
this.log.exit("loadShape", true);

return true;
} catch (error) {
  this.loading.delete(type);
  this.failed.set(type, error);

  this.log.error(`Failed to load shape '${type}':`, error);

  if (this.throwOnError) {
    throw error;
  }

  this.log.exit("loadShape", false);
  return false;
}
}

```

Complete Shape Class Example

```

// CircleShape.js
import { BaseShape } from "../../base/BaseShape.js";

export class CircleShape extends BaseShape {
  constructor(config) {
    super(config);
    // Config is now available as:

```

```
// this.id, this.name, this.defaultSize, this.defaultStyle, etc.
}

render(x, y, width, height, styleOverride = {}) {
  // Merge config defaults with runtime overrides
  const style = { ...this.defaultStyle, ...styleOverride };

  // Create SVG
  const g = document.createElementNS("http://www.w3.org/2000/svg", "g");
  const circle = document.createElementNS(
    "http://www.w3.org/2000/svg",
    "circle"
  );

  const cx = x + width / 2;
  const cy = y + height / 2;
  const r = Math.min(width, height) / 2;

  circle.setAttribute("cx", cx);
  circle.setAttribute("cy", cy);
  circle.setAttribute("r", r);
  circle.setAttribute("fill", style.fill);
  circle.setAttribute("stroke", style.stroke);
  circle.setAttribute("stroke-width", style.strokeWidth);

  g.appendChild(circle);
  return g;
}

getPortPositions(x, y, width, height) {
  const cx = x + width / 2;
  const cy = y + height / 2;
  const r = Math.min(width, height) / 2;

  return this.defaultPorts.map((port) => {
    let px, py;
    switch (port.side) {
      case "top":
        px = cx;
        py = cy - r;
        break;
      case "right":
        px = cx + r;
        py = cy;
        break;
      case "bottom":
        px = cx;
        py = cy + r;
        break;
      case "left":
        px = cx - r;
        py = cy;
        break;
    }
  })
}
```

```
        return { ...port, x: px, y: py };
    });
}
}

export default CircleShape;
```

Complete Usage Example

```
// main.js
import { ShapeLoader } from "../shapes/loader/ShapeLoader.js";
import { ShapeRegistry } from "../shapes/registry/ShapeRegistry.js";

async function initializeApp() {
    // 1. Create registry
    const shapeRegistry = new ShapeRegistry();

    // 2. Create loader
    const loader = new ShapeLoader(shapeRegistry, {
        validateOnLoad: true,
        throwOnError: false,
    });

    // 3. Load built-in shapes
    console.log("Loading built-in shapes...");
    const results = await loader.loadBuiltInShapes("/src/shapes/library");

    console.log(`Loaded: ${results.loaded} shapes`);
    console.log(`Failed: ${results.failed} shapes`);

    if (results.failed > 0) {
        console.error("Some shapes failed to load:", results.categories);
    }

    // 4. Use a shape
    const shapeDef = shapeRegistry.getShape("basic-circle");
    const circleShape = shapeDef.createInstance();

    // 5. Render shape
    const svg = circleShape.render(100, 100, 80, 80, {
        fill: "#FF0000",
        stroke: "#000000",
    });

    // 6. Add to DOM
    document.getElementById("canvas").appendChild(svg);

    console.log("App initialized successfully!");
}
```

```
initializeApp();
```

End of Documentation